RetryRL: Reinforcement Learning for Intelligent API Recovery

Himanshu Shekhar (B23MT1021) Amrutanshu Sahoo (B23MT1009)
GitHub: https://github.com/HimanshuShekhar12/RetryRL-API-Recovery

1 Problem and Approach

When an API call fails, a client must decide how to recover: *retry* immediately, *wait* (back off), serve a *cached* response, or *switch to a fallback* service. Each choice trades reliability, latency and cost, and failures arrive in bursts, so no fixed rule is best everywhere. We model this as a Markov Decision Process (MDP) and learn the policy with model-free temporal-difference RL (Q-learning and SARSA), compared against rule-based policies used *only as benchmarks*.

2 RL Formulation

MDP $(\mathcal{S}, \mathcal{A}, P, R, \gamma)$ with $\gamma = 0.95$. One episode is one request that has already failed once; it ends on delivery or after 8 decisions (abandoned). The transition kernel P is hidden from the agent.

State space $\mathcal{S}$ (discrete, $|\mathcal{S}| = 3 \cdot 3 \cdot 3 \cdot 4 \cdot 4 \cdot 2 = 864$). The agent observes only symptoms, never the true service health:

Feature	Levels	Definition
Last outcome	3	timeout / 5xx / 429
Failure-rate EMA	3	< 0.5 / 0.5–0.8 / ≥ 0.8
Latency EMA	3	< 1.5 / 1.5–3 / ≥ 3 s
Cache freshness	4	none / fresh / aging / stale
Failed attempts	4	1 / 2 / 3 / 4+
Fallback tried	2	no / yes

The two EMAs are running client statistics over all traffic, so they carry information about the hidden health.

Action space $\mathcal{A}$ (discrete, 4): `RETRY` (immediate re-call), `WAIT` (back off, then re-call), `USE_CACHE` (terminal), `FALLBACK` (call the secondary service).

Reward R**.** With latency L_t (s) and monetary cost C_t of step t:

$$R_t = r_{\text{src}}\mathbf{1}_{\text{ok}} - 0.5L_t - 1.0C_t - 3\,\mathbf{1}_{429} - 3\,\mathbf{1}_{\text{inv}} - 10\,\mathbf{1}_{\text{abandon}} \quad (1)$$

where $r_{\text{src}} = 10$ for a primary success, 8.5 for the fallback (lower quality), and $8 - 5\min(1, a/40)$ for a cache hit of age a s (stale data is penalised). The terms reward reliability, penalise slow and expensive recovery, penalise hammering a rate-limited API (429), penalise calling an empty cache (inv), and penalise abandoning a request.

3 Environment and Algorithms

Simulator (custom Gymnasium environment, written from scratch). Primary-service health follows a 3-state Markov chain (Healthy/Degraded/Outage) with rows [.90, .07, .03], [.12, .78, .10], [.04, .08, .88], so outages persist. Per-call success probability is .95/.50/.03 by health; failures are timeouts (5 s), 5xx or 429. Success latency is lognormal. Two or more back-to-back retries raise 429 risk, which `WAIT` resets. The fallback costs 2.5 (vs. 0.1) and succeeds with probability .90. The cache has a random age. For *robustness* we also test an unseen **harsh** regime with longer, more frequent outages (rows [.80, .12, .08], [.08, .72, .20], [.02, .05, .93]).

Algorithms. Both are tabular, model-free and ϵ-greedy:

$$\text{Q-learning: } Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma \max_{a'} Q(s',a') - Q(s,a)]$$

$$\text{SARSA: } Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma Q(s',a') - Q(s,a)]$$

We use $\alpha = 0.05$, 200,000 episodes, ϵ decayed exponentially from 1.0 to 0.05 (reached at 60% of training), and 5 seeds per algorithm. The deployed policy is greedy, $\pi(s) = \arg\max_a Q(s,a)$. We chose tabular methods because $|\mathcal{S}| = 864$ is small and the table is directly interpretable.

Baselines (comparison only, not RL): AlwaysRetry, ExpBackoff (wait, then fallback), CacheFirst, ThresholdFallback (retry twice, then fallback), and Random.

4 Results

All agents are evaluated greedily for 10,000 episodes (mean ± std over 5 training seeds).

Table 1: Average reward and latency (s). Success rate is 100% for every method except AlwaysRetry (88%/61%) and Random (99%) in harsh.

	Normal		Harsh (unseen)	
Method	Reward	Lat.	Reward	Lat.
AlwaysRetry	3.38	7.42	−7.58	13.96
ExpBackoff	5.39	9.79	1.48	15.13
CacheFirst	5.39	3.76	4.62	4.71
ThresholdFallback	6.85	6.01	4.07	9.07
Random	5.18	5.63	3.19	7.71
Q-learning	**6.99**±.04	4.12	**5.84**±.01	4.44
SARSA	6.97±.04	4.07	5.81±.01	4.43

Findings. In the training regime both agents beat every baseline, but the reward gain over ThresholdFallback is modest (+0.14, about 2%, roughly 3σ across seeds); the clearer win is latency, about 31% lower. On the unseen harsh regime the gap is large: +26% reward over the best baseline (CacheFirst) and about half the latency of ThresholdFallback, with lower variance (Fig. 3). Q-learning and SARSA perform almost identically.

The learned policy adapts. Although true health is hidden, the agent infers it from symptoms (Fig. 4): under

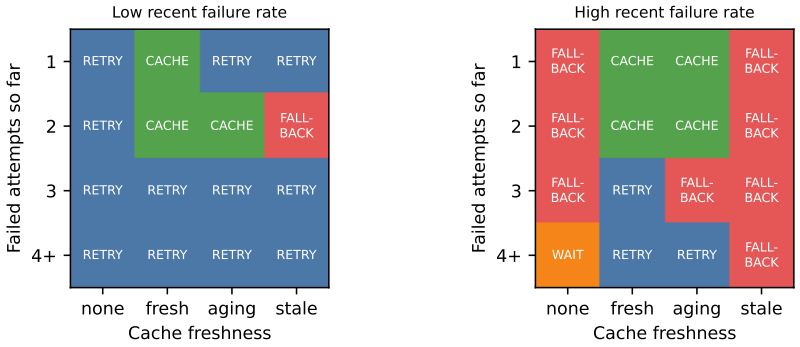


Figure 1: Greedy action by cache freshness and failed attempts so far (Q-learning, seed 0), for a low (left) and high (right) recent failure rate.

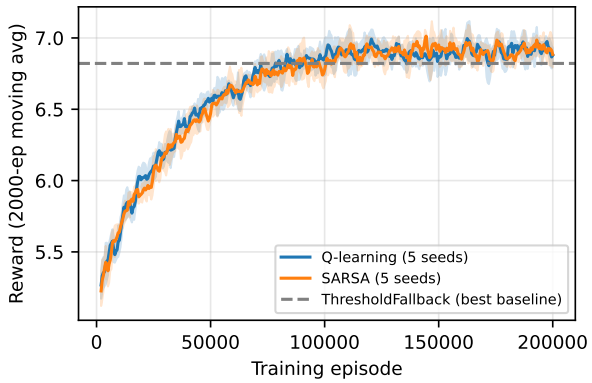


Figure 2: Training curves (mean ± std over 5 seeds). Both agents rise above the best baseline.

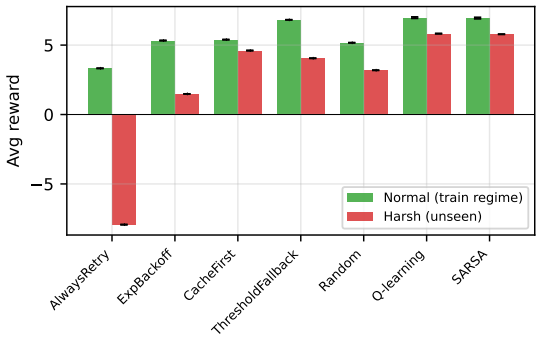


Figure 3: Average reward in the training (normal) and unseen harsh regime.

a Healthy service it retries 85% of the time, while in an Outage it retries only 17% and shifts to cache (27%) and fallback (56%); the baseline's mix barely changes. The policy map (Fig. 1) shows the same: with a low recent failure rate it retries, with a high one it switches to cache or fallback.

Design lesson. An early SARSA seed scored poorly at evaluation despite a normal training reward: three actions had nearly tied Q-values and the greedy choice was `USE_CACHE` with an empty cache, an invalid action that leaves the state unchanged, so the greedy policy looped until abandonment (ϵ-exploration hid this during training).

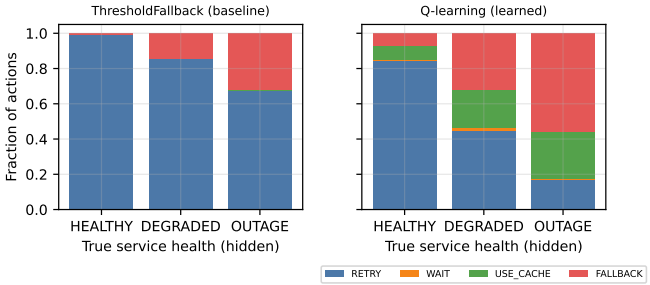


Figure 4: Action mix by true (hidden) service health.

Raising the empty-cache penalty from 1 to 3 fixed it for all seeds.

5 Limitations and Conclusion

The environment is a simulation, and the state is only a partial view of the hidden service health, which limits the achievable reward. Future work includes function approximation (DQN) for richer observations and real failure traces. We showed that API recovery is naturally an MDP and that tabular Q-learning and SARSA learn a health-adaptive policy that beats fixed rules, especially under unseen, harsher failure conditions.

References. [1] R. Sutton and A. Barto, *Reinforcement Learning: An Introduction*, 2nd ed., MIT Press, 2018. [2] C. Watkins and P. Dayan, "Q-learning," *Machine Learning*, 1992. [3] G. Rummery and M. Niranjan, "On-line Q-learning using connectionist systems," 1994.